CY Cergy Paris Université

Licence 2 Informatique — Génie Logiciel et Projet

Simulation Multi-Agents de Souris sur une Grille 2D



Autrices :

SIMYA Meriem LAIFAOUI Ines ANOUD Salma

Encadrant : M. Tianxiao LIU

Année universitaire 2025–2026

Table des matières

Remerciements **4**

1 Introduction **4**
1.1 Contexte académique . . . 4
1.2 Présentation du sujet . . . 4
1.3 Objectifs du projet . . . 4
1.4 Fonctionnalités implémentées . . . 5
1.5 Structure du rapport . . . 5

2 Spécification du projet **5**
2.1 Vue d'ensemble du système . . . 5
2.2 La grille de simulation . . . 6
2.3 Les agents : les souris . . . 6
2.4 Les quatre caractères . . . 6
2.5 Les types de nourriture et d'obstacles . . . 7
2.6 L'algorithme de déplacement — 8 niveaux de priorité . . . 8
2.7 Les interactions sociales . . . 8
2.7.1 Le partage Donneur → Receveur . . . 8
2.7.2 La reproduction . . . 9
2.8 La mémoire des souris . . . 9
2.9 La cible dynamique : le trésor . . . 10

3 Réalisation technique **11**
3.1 Architecture générale du projet . . . 11
3.2 Organisation des packages . . . 11
3.3 Diagrammes de classes . . . 12
3.3.1 Diagramme simplifié — relations entre classes . . . 13
3.3.2 Diagramme complet — attributs et méthodes . . . 13
3.4 Les design patterns appliqués . . . 14
3.4.1 Pattern Template Method (tree_v1) — `nextRound()` . . . 14
3.4.2 Pattern Strategy (tree_v2) — `PaintStrategy` . . . 15
3.4.3 Pattern Builder (tree_v1) — `GameBuilder` . . . 15
3.4.4 Pattern Singleton (tree_v1) — `JournalFenetre` . . . 15
3.4.5 Pattern Simple Factory (tree_v1) — `addFood()`, `addObstacle()` . . . 15
3.5 La boucle principale de simulation . . . 16
3.6 Le moteur de simulation : `Environnement.java` . . . 16
3.6.1 Le déplacement des souris . . . 16
3.6.2 La gestion des interactions . . . 17
3.6.3 Le calcul du score de colonisation . . . 17
3.7 Le système de mémoire en détail . . . 18
3.8 Le système d'interactions et les flashes visuels . . . 18
3.9 La cible dynamique : implémentation du trésor . . . 19
3.9.1 Algorithme de placement . . . 19
3.9.2 Rendu visuel du trésor . . . 20
3.9.3 Les trois écrans de fin . . . 20
3.10 L'IHM Graphique en détail . . . 20
3.10.1 Vue globale de la fenêtre principale . . . 20
3.10.2 Détail : le StatsPanel . . . 20
3.10.3 Détail : le GameDisplay . . . 21
3.10.4 Détail : la JournalFenetre . . . 21

3.11 Journalisation avec Log4j 22

4 Manuel utilisateur **23**
4.1 Environnement de développement 23
4.2 Configuration initiale de la simulation 23
4.3 Contrôle de la simulation en cours 24
4.4 Lecture des statistiques 24
4.5 Consulter le journal d'une souris 24
4.6 Suivre l'évolution du trésor 25
4.7 Interprétation des fins de simulation 25

5 Déroulement du projet **25**
5.1 Organisation du travail en équipe 25
5.2 Problèmes rencontrés et solutions apportées 25
5.2.1 Problème 1 : Conflit de versions Java dans Eclipse 26
5.2.2 Problème 2 : Images des sprites non trouvées au lancement 26
5.2.3 Problème 3 : Fichiers de configuration Log4j non trouvés 26
5.2.4 Problème 4 : Boucles infinies de déplacement des souris 26
5.2.5 Problème 5 : Souris qui heurtent les mêmes obstacles répétitivement 27
5.2.6 Problème 6 : Fichier GameDisplay.java vidé par erreur d'encodage 27
5.2.7 Problème 7 : Trésor inaccessible (entouré d'obstacles) 27
5.2.8 Problème 8 : Redémarrage automatique indésirable 27
5.3 Les tests JUnit 28

6 Conclusion **28**
6.1 Bilan technique 29
6.2 Difficultés surmontées 29
6.3 Perspectives d'amélioration 30
6.4 Apprentissages 30

Table des figures

1 Architecture en cinq couches du projet 12
2 Diagramme de classes simplifié — relations structurelles entre les 17 classes 13
3 Diagramme de classes complet — attributs et méthodes de toutes les classes 14
4 Protocole complet du partage Donneur → Receveur 19
5 Vue globale de la fenêtre principale — MainGUI (BorderLayout) 21
6 Détail du StatsPanel — 5 blocs mis à jour à chaque tour 21
7 Détail du GameDisplay (overlay config. à gauche, grille de simulation à droite) 22
8 Détail de la JournalFenetre (fenêtre contextuelle par souris) 22

Liste des tableaux

1 Paramètres de configuration de la simulation 6
2 Attributs complets d'une souris 7
3 Comportements des quatre caractères 7
4 Types de nourriture et gains d'énergie 8
5 Types d'obstacles et pénalités 8
6 Algorithme de déplacement — 8 niveaux de priorité décroissante 9
7 Attributs de la classe Memoire 10
8 Chronologie complète du trésor 10
9 Les trois fins de simulation possibles 11
10 Organisation des sept packages du projet 12

11 Types d'interactions et leurs représentations visuelles 18
12 Paramètres de configuration disponibles au démarrage 23
13 Boutons de contrôle et leurs actions 24
14 Répartition du travail au sein de l'équipe 26
15 Classes de tests JUnit et cas couverts 28
16 Métriques du projet 29

Remerciements

Les autrices souhaitent remercier vivement **M. Tianxiao LIU**, enseignant-chercheur à CY Cergy Paris Université, pour son encadrement bienveillant et rigoureux tout au long du cours de Génie Logiciel et Projet (GLP).

Nous le remercions particulièrement pour :

- Les **cinq programmes de démonstration** qui nous ont servi de modèles architecturaux :
  - *Chronometer* (TD introductif — séparation IHM/Moteur)
  - *tree_ v1* (Builder, Template Method, Singleton, Simple Factory)
  - *tree_ v2* (Strategy pattern — PaintStrategy)
  - *tree_ v3* (Log4j, structure finale du projet)
  - *Aircraft* et *Train* (simulation multi-agents)
- L'**exemple de rapport LaTeX** pour la structure de ce document.
- Les **cinq librairies** mises à disposition :
  - `log4j-1.2.17.jar` — journalisation (TD 6)
  - `jfreechart-1.0.19.jar` et `jcommon-1.0.23.jar` — graphiques
  - `junit-4.11.jar` et `hamcrest-core-1.3.jar` — tests automatisés (TD 5)
- Les **slides de cours**, les TDs CNP, les ateliers de tests et de journalisation.
- Sa **disponibilité** pour répondre à nos questions lors des séances.

1. Introduction

1.1. Contexte académique

Ce projet s'inscrit dans le cadre du cours de **Génie Logiciel et Projet (GLP)** de Licence 2 Informatique à CY Cergy Paris Université, encadré par M. Tianxiao LIU. Il représente l'aboutissement de l'ensemble des notions vues en cours et en TD : conception orientée objet, design patterns, séparation IHM/Moteur, tests automatisés, journalisation et documentation.

Le cours GLP s'appuie sur une progression pédagogique construite autour de cinq programmes de démonstration fournis par M. Liu : *Chronometer* (introduction à la séparation IHM/Moteur), *tree_ v1* (design patterns Builder, Template Method, Singleton, Simple Factory), *tree_ v2* (pattern Strategy), *tree_ v3* (journalisation Log4j dans la structure finale), et *Aircraft*/ *Train* (simulation multi-agents avec threads). Notre projet intègre et dépasse l'ensemble de ces démonstrations.

1.2. Présentation du sujet

Le sujet consiste à concevoir et implémenter une **simulation multi-agents** dans laquelle des souris autonomes évoluent sur une grille 2D. Chaque souris est un agent indépendant qui perçoit son environnement local, prend des décisions selon ses attributs internes, interagit avec les autres agents et adapte son comportement en fonction de ce qu'elle mémorise.

La simulation comporte plusieurs niveaux de complexité croissante. Au niveau le plus simple, les souris se déplacent aléatoirement et mangent pour survivre. À un niveau intermédiaire, elles interagissent socialement (partage d'informations, reproduction, amitié), développent une mémoire des lieux et des relations, et adoptent des comportements stratégiques selon leur caractère. Au niveau le plus avancé, une **cible dynamique** (le trésor) introduit une mécanique de jeu progressive avec une condition de victoire, une fenêtre temporelle et trois fins de simulation possibles.

1.3. Objectifs du projet

Les objectifs techniques du projet sont les suivants :

- Concevoir une architecture logicielle en couches séparées, respectant le principe de séparation IHM/Moteur.

— Appliquer au moins cinq design patterns issus du cours .
— Implémenter une simulation multi-agents avec comportements émergents.
— Produire une interface graphique interactive et informative en Java Swing.
— Écrire des tests automatisés avec JUnit 4 couvrant les composants principaux.
— Journaliser les événements de la simulation avec Log4j.
— Documenter le projet avec ce rapport LaTeX suivant les normes du cours.

1.4. Fonctionnalités implémentées

Le projet final implémente les fonctionnalités suivantes :
— Une **grille 2D configurable** (5 à 40 colonnes, 5 à 30 lignes) avec obstacles, nourriture et agents.
— Trois types d'**obstacles** : bûche, rivière, rocher.
— Trois types de **nourriture** : fromage (+25 énergie), pomme (+35 énergie), graines (+15 énergie).
— Des **souris autonomes** avec 4 caractères, 2 sexes, 3 couleurs, une énergie et une mémoire.
— Un **algorithme de déplacement** à 8 niveaux de priorité avec anti-boucle (historique 6 positions).
— Un **système de mémoire** : cases de nourriture connues, cases dangereuses, relations sociales, cibles.
— Des **interactions sociales** : partage D→R (vraie ou fausse info), reproduction, amitié, piège.
— Des **flashes visuels** pour chaque type d'interaction (bulle, cœur, nuage rouge).
— Des **cycles évolutifs** : jusqu'à 5 générations de souris avec extinction et renouvellement.
— Une **cible dynamique** (trésor) avec fenêtre temporelle 100–300 tours et 3 fins de simulation.
— Un **score de colonisation** calculé à chaque tour (formule pondérée sur 100 points).
— Une **IHM complète** : grille animée, StatsPanel dynamique, JournalFenetre par souris, 8 boutons.
— Des **tests JUnit** sur 5 classes couvrant tous les composants principaux.
— Une **journalisation** Log4j en console, en fichier texte et en fichier HTML.

1.5. Structure du rapport

Ce rapport est organisé comme suit :
— **Section 2 — Spécification** : description détaillée des règles, attributs et comportements.
— **Section 3 — Réalisation technique** : architecture, design patterns, algorithmes, code et schémas.
— **Section 4 — Manuel utilisateur** : guide d'utilisation du logiciel.
— **Section 5 — Déroulement du projet** : organisation, problèmes rencontrés, tests.
— **Section 6 — Conclusion** : bilan, perspectives et apprentissages.

2. Spécification du projet

2.1. Vue d'ensemble du système

La simulation modélise un **écosystème artificiel** dans lequel des agents autonomes — les souris — évoluent librement sur une grille 2D discrète. Le temps est discret : à chaque *tour*, chaque agent effectue une action (déplacement, interaction, reproduction). La simulation est observable en temps réel via une interface graphique interactive.

Le système repose sur trois types d'entités principales. Les **agents mobiles** comprennent les souris adultes et les souriceaux. Les **entités fixes** comprennent les obstacles (bûches, rivières, rochers), les nourritures (fromage, pomme, graines) et le trésor. Enfin, les **entités transitoires** sont les flashes d'interaction, affichés pendant exactement 3 tours avant d'être supprimés automatiquement.

La simulation est entièrement paramétrable au démarrage : l'utilisateur choisit la taille de la grille, le nombre initial de souris, de nourritures, d'obstacles et la population maximale autorisée. Ces paramètres influencent directement la dynamique de la simulation.

2.2. La grille de simulation

La grille est une matrice de dimensions `GRID_LINES` × `GRID_COLS`, configurables entre 5 et 40 colonnes et entre 5 et 30 lignes. Chaque case est identifiée par ses coordonnées (ligne, colonne) et peut contenir une souris, un obstacle, une nourriture ou le trésor. Une case peut également être vide — on parle alors de case libre.

Le tableau 1 récapitule les paramètres de configuration par défaut et leurs plages de valeurs autorisées.

Paramètre	**Valeur par défaut**	**Plage autorisée**
Nombre de colonnes	25	5 – 40
Nombre de lignes	17	5 – 30
Nombre initial de souris	20	1 – 30
Nombre initial de nourritures	18	1 – 30
Nombre initial d'obstacles	5	0 – 20
Population maximale	40	5 – 80
Énergie initiale d'une souris	80	fixe
Énergie maximale	100	fixe
Perte d'énergie par tour	1	fixe
Perte d'énergie sur obstacle	10	fixe

TABLE 1 – Paramètres de configuration de la simulation

2.3. Les agents : les souris

Chaque souris est une instance de la classe `Souris`, caractérisée par un ensemble d'attributs fixes (identifiant, sexe, caractère, couleur) et d'attributs dynamiques (énergie, âge, position, mémoire). Le tableau 2 présente l'ensemble de ces attributs.

Cycle de vie d'une souris. Une souris naît avec une énergie de 80 points. À chaque tour, elle perd 1 point d'énergie. Lorsqu'elle arrive sur une case contenant de la nourriture, elle la consomme et regagne un certain nombre de points selon le type de nourriture. Si son énergie atteint 0, elle meurt et est retirée de la grille. Une souris peut également se reproduire si elle rencontre une souris de sexe opposé dans les conditions requises, ce qui produit un souriceau.

2.4. Les quatre caractères

Le caractère est l'attribut le plus déterminant du comportement social d'une souris. Il est attribué aléatoirement à la naissance et ne peut pas changer. Il définit le rôle de la souris lors des interactions Donneur/Receveur et son rapport à l'information.

Attribut	Type	Valeur initiale	Description
`id`	`int`	incrémental	Identifiant unique global
`sexe`	`Sexe`	aléatoire	MALE ou FEMELLE
`caractere`	`Caractere`	aléatoire	SINCERE, SOCIABLE, EGOISTE, MECHANTE
`couleur`	`CouleurRobe`	aléatoire	GRIS, BLANC, MARRON
`energie`	`int`	80	Décroît de 1 par tour, 0 = mort
`age`	`int`	0	Incrémenté chaque tour
`position`	`Case`	aléatoire	Case courante sur la grille
`memoire`	`Memoire`	vide	Cases connues, relations, cibles
`vivante`	`boolean`	true	Passe à false si énergie = 0

TABLE 2 – Attributs complets d'une souris

Caractère	Comportement en tant que Donneur	Comportement en tant que Receveur
SINCERE	Partage la vraie position de nourriture lorsque son énergie est pleine	Fait confiance et suit l'information reçue
SOCIABLE	Partage si énergie ≥ 70, parfois avec de légères imprécisions	Suit l'information reçue
EGOISTE	Ne partage jamais d'information, garde ses connaissances pour elle	Accepte l'information mais ne la suit pas
MECHANTE	Donne toujours une fausse position pour piéger le Receveur	Ignore les informations reçues

TABLE 3 – Comportements des quatre caractères

Le tableau 3 met en évidence la diversité des comportements sociaux. Une souris SINCERE contribue positivement à la colonie en partageant des informations fiables. Une souris MECHANTE, en revanche, peut entraîner ses congénères vers des pièges, créant des interactions négatives visualisées par un flash rouge "Oh ! Un piège !".

2.5. Les types de nourriture et d'obstacles

Trois types de nourriture sont disponibles dans la simulation, chacun apportant un gain d'énergie différent. Le tableau 4 présente leurs caractéristiques.

Lorsque le bouton `+ Nourriture` est pressé, un type est choisi aléatoirement parmi les trois, avec une probabilité égale de $\frac{1}{3}$ pour chacun. La nourriture est placée sur une case libre aléatoire de la grille.

Trois types d'obstacles sont également disponibles, représentés par des sprites PNG sans fond blanc. Le tableau 5 en présente les caractéristiques.

Type	Énergie gagnée	Représentation visuelle
Fromage	+25 points	Sprite PNG fromage jaune
Pomme	+35 points	Sprite PNG pomme rouge
Graines	+15 points	Sprite PNG graines

TABLE 4 – Types de nourriture et gains d'énergie

Type	Perte si collision	Représentation visuelle
Bûche	−10 énergie	Sprite bûche marron (image PNG)
Rivière	−10 énergie	Sprite rivière bleue (image PNG)
Rocher	−10 énergie	Sprite obstacle (image PNG)

TABLE 5 – Types d'obstacles et pénalités

Une souris qui heurte un obstacle perd 10 points d'énergie et mémorise immédiatement la case comme dangereuse. Ce mécanisme garantit qu'une souris ne retombe **jamais deux fois** sur le même obstacle : la case est ajoutée à `Memoire.casesDangereuses` et sera systématiquement évitée (priorité 7 de l'algorithme de déplacement).

2.6. L'algorithme de déplacement — 8 niveaux de priorité

À chaque tour, le moteur appelle `choisirProchaineCaseAvecAttraction()` pour chaque souris vivante. Cette méthode est le cœur de l'intelligence des agents : elle évalue 8 conditions dans l'ordre décroissant de priorité et retourne la meilleure case candidate. Le tableau 6 décrit l'ensemble des priorités.

Cet algorithme garantit que les souris adoptent un comportement cohérent et prévisible. Une souris qui connaît le trésor et a l'énergie suffisante y fondera directement. Une souris affamée ignorera le trésor pour chercher de la nourriture. Une souris qui a reçu une fausse information d'une souris MECHANTE suivra quand même la cible — jusqu'à l'obstacle.

Anti-boucle de déplacement. Sans précaution, les souris peuvent se retrouver bloquées dans des boucles entre deux cases. Pour éviter ce phénomène, un historique des 6 dernières positions est maintenu pour chaque souris. Avant de choisir une case candidate, les cases récemment visitées sont exclues de la liste des candidats. Ce mécanisme est implémenté via une `Deque<String>` par souris dans `historiquePositions`.

2.7. Les interactions sociales

Les interactions sociales constituent la richesse comportementale de la simulation. Elles surviennent automatiquement quand deux souris se trouvent sur des cases adjacentes.

2.7.1. Le partage Donneur → Receveur

L'interaction la plus importante est le partage d'information entre un Donneur et un Receveur. Le protocole est le suivant : le Donneur vérifie s'il a l'énergie pleine et s'il connaît une case de nourriture. Si oui, et si la paire n'a pas déjà échangé (mémorisé dans l'ensemble `pairesDejaVues`), il transmet la position de la nourriture au Receveur. Ce dernier la mémorise dans `caseCiblePartage` et s'y dirige dès le tour suivant (priorité 1).

Priorité	Condition	Action résultante
0	Trésor actif, tour $\geq$ 100, énergie $\geq$ 85, âge $\geq$ 15	Se diriger vers le trésor (priorité absolue)
1	Cible partagée non nulle (`caseCiblePartage`)	Suivre la cible aveuglément, vraie ou fausse
2	Cible nourriture mémorisée (`caseCibleNourriture`)	Se diriger vers la nourriture connue
3	Cases nourriture mémorisées non vides	Retourner à une case de nourriture connue
3.5	Trésor visible dans un rayon de 2 cases	Se diriger vers le trésor (découverte exploration)
4	Nourriture visible dans un rayon de 5 cases	Se diriger vers la nourriture visible
5	$\geq$ 3 souris voisines (surpopulation locale)	Fuir vers une case moins dense
6	Souris de sexe opposé visible	Attraction aléatoire (probabilité 35%)
7	Cases dangereuses dans la mémoire	Évitement systématique
8	Aucune condition précédente satisfaite	Déplacement aléatoire parmi les cases libres

TABLE 6 – Algorithme de déplacement — 8 niveaux de priorité décroissante

Si le Donneur est MECHANTE, il transmet une fausse position. Le Receveur, ne sachant pas distinguer une vraie d'une fausse information, la suit aveuglément. Si la case indiquée contient un obstacle, un flash PIEGE rouge "Oh ! Un piège !" s'affiche sur la souris piégée, qui perd 10 points d'énergie supplémentaires et mémorise la case comme dangereuse.

2.7.2. La reproduction

Deux souris de sexe opposé sur des cases adjacentes peuvent se reproduire selon la probabilité définie par l'équation 1. Lorsque la reproduction a lieu, un souriceau est créé sur une case libre adjacente. Il hérite de la couleur d'un parent (tirage aléatoire), reçoit un caractère aléatoire et naît avec l'énergie initiale de 80 points. Un flash REPRODUCTION (cœur ♡) est affiché entre les deux parents. L'événement est journalisé dans le journal de chacun des deux parents et du souriceau.

$$P(\text{repro}) = 0{,}20 + \frac{\bar{E}}{200} \pm \Delta_{car} \tag{1}$$

où $\bar{E}$ est l'énergie moyenne des deux parents normalisée sur 100, et Δ_{car} est un bonus ou malus selon le caractère ($+0.05$ pour SINCERE, -0.05 pour MECHANTE).

2.8. La mémoire des souris

La mémoire est l'un des composants les plus importants de la simulation. Elle permet à chaque souris d'apprendre de ses expériences passées et d'adopter des comportements de plus en plus sophistiqués au fil du temps. Elle est représentée par la classe `Memoire`, dont le tableau 7 décrit les attributs.

Attribut	Type	Rôle dans le comportement
`casesNourritureConnues`	`List<Case>`	Cases où de la nourriture a déjà été trouvée (priorité 3)
`casesDangereuses`	`List<Case>`	Cases avec obstacle mémorisé, évitées systématiquement (priorité 7)
`relationSociales`	`List<RelationSociale>`	Souris connues (amitié, reproduction), condition trésor
`caseCiblePartage`	`Case`	Cible reçue d'un Donneur (priorité 1, vraie ou fausse)
`caseCibleNourriture`	`Case`	Cible nourriture mémorisée (priorité 2)

TABLE 7 – Attributs de la classe Memoire

La mémoire est enrichie à chaque événement. Quand une souris trouve de la nourriture, la case est ajoutée à `casesNourritureConnues`. Quand elle heurte un obstacle, la case est ajoutée à `casesDangereuses`. Quand elle reçoit une information d'un Donneur, la case est mémorisée dans `caseCiblePartage`. Quand elle rencontre une autre souris, une `RelationSociale` est créée et ajoutée à `relationSociales`.

Importance pour le trésor. La mémoire joue un rôle crucial dans la condition d'accès au trésor. Une souris doit avoir mémorisé au moins 5 cases de nourriture et entretenir au moins 2 relations sociales pour pouvoir l'obtenir. Ces conditions garantissent que seules les souris ayant suffisamment exploré et interagi peuvent gagner.

2.9. La cible dynamique : le trésor

Le trésor est la fonctionnalité la plus originale du projet. Il introduit une mécanique de jeu progressive : une cible cachée qui devient accessible après un certain nombre de tours, sous des conditions strictes, et qui entraîne la fin de la simulation si elle est atteinte.

Tour	Événement
50	Le trésor apparaît sur le bord de la grille. Il est grisé, avec un cadenas doré et un compte à rebours "T-X" visible.
100	Le trésor s'illumine en doré avec un halo pulsant animé. La course commence.
100–300	Fenêtre de recherche active. Les souris éligibles cherchent activement le trésor.
300	La fenêtre se ferme. La simulation continue jusqu'à extinction de la colonie.
Extinction	SAD END : toutes les souris mortes sans trésor. Relance avec 3 nouvelles souris.

TABLE 8 – Chronologie complète du trésor

Conditions d'obtention. Pour obtenir le trésor, une souris doit remplir simultanément les quatre conditions suivantes au moment où elle arrive sur la case :

- Énergie ≥ 85 points sur 100 (être en bonne santé).
- Âge ≥ 15 tours (avoir de l'expérience).
- Mémoire contenant au moins 5 cases de nourriture connues (avoir bien exploré).
- Au moins 2 relations sociales mémorisées (avoir interagi avec la colonie).

Si une souris arrive sur le trésor sans remplir toutes ces conditions, elle reçoit un message explicatif dans son journal selon la condition manquante, et sa cible trésor est effacée. Elle devra se préparer avant de réessayer.

Les trois fins de simulation. La simulation peut se terminer de trois façons distinctes, chacune donnant lieu à un écran de fin spécifique :

Type de fin	**Condition**	**Écran affiché**
Victoire	Une souris atteint le trésor entre les tours 100 et 300	Écran doré avec identité de la gagnante
SAD END	Toutes les souris meurent sans avoir trouvé le trésor	Écran rouge sombre, relance avec 3 souris
Colonisation optimale	Le score de colonisation se stabilise pendant 10 tours consécutifs	Écran avec note D à S+ et barre de score

TABLE 9 – Les trois fins de simulation possibles

Le score de colonisation (équation 2) mesure la progression de la colonie à chaque tour. Il est calculé comme une somme pondérée de quatre critères : la couverture de la grille (C), l'énergie moyenne normalisée (E), la diversité des zones occupées dans les 4 quadrants (D) et la taille relative de la population (P).

$$\text{Score} = 40 \times C + 30 \times E + 20 \times D + 10 \times P \tag{2}$$

3. Réalisation technique

3.1. Architecture générale du projet

Le projet suit une architecture en **couches séparées**, conforme aux principes enseignés en cours GLP, inspirée directement des programmes *tree_ v3* et *Aircraft* fournis par M. Liu. La règle fondamentale est que chaque couche ne dépend que de la couche immédiatement inférieure. Aucun composant d'une couche basse ne connaît les composants d'une couche haute.

L'interface `MobileInterface` joue un rôle clé dans ce découplage : elle définit le contrat entre l'IHM et le moteur, permettant à `MainGUI` d'utiliser `Environnement` sans en connaître les détails d'implémentation.

3.2. Organisation des packages

Le projet est organisé en sept packages, chacun ayant une responsabilité clairement définie. Le tableau 10 présente cette organisation.

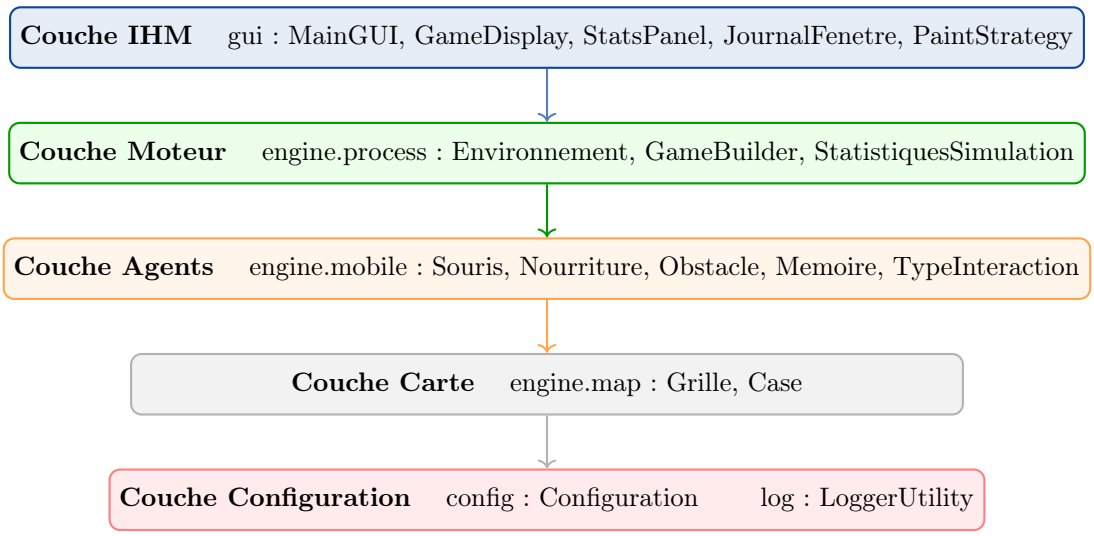


FIGURE 1 – Architecture en cinq couches du projet

Package	Contenu et responsabilité
`config`	`Configuration.java` — toutes les constantes de la simulation (énergies, seuils, dimensions)
`engine.map`	`Grille.java` et `Case.java` — modèle de la grille 2D et de ses cases
`engine.mobile`	`Souris.java`, `Nourriture.java`, `Obstacle.java`, `Memoire.java`, `TypeInteraction.java`, `FlashInteraction.java`, `Caractere.java`, `CouleurRobe.java`, `Sexe.java`, `TypeNourriture.java`, `TypeObstacle.java`, `RelationSociale.java`
`engine.process`	`Environnement.java` (moteur principal), `GameBuilder.java`, `MobileInterface.java`, `StatistiquesSimulation.java`
`gui`	`MainGUI.java`, `GameDisplay.java`, `PaintStrategy.java`, `StatsPanel.java`, `JournalFenetre.java`, `JournalPanel.java`
`log`	`LoggerUtility.java`, `log4j-text.properties`, `log4j-html.properties`
`test`	`TestGame.java`, `TestSouris.java`, `TestInteractions.java`, `TestEnvironnement.java`, `TestSuiteAll.java`

TABLE 10 – Organisation des sept packages du projet

3.3. Diagrammes de classes

Le projet comprend 17 classes réparties en cinq couches. Nous présentons d'abord le diagramme simplifié pour visualiser les grandes relations, puis le diagramme complet avec tous les attributs et méthodes.

3.3.1. Diagramme simplifié — relations entre classes

La figure 2 représente les relations structurelles entre les 17 classes du projet. On y distingue les quatre grandes familles de liens :

- **Composition** (losange plein) : `Environnement` compose une `Grille` et une liste de `Souris` ; `Grille` compose un tableau de `Case` ; `Souris` compose une `Memoire` et un `Journal` (1 pour 1).
- **Agrégation** (losange vide) : `Memoire` agrège des `RelationSociale` et des `Interaction` (0..*) ; `Journal` agrège des `EntreeJournal` (1..*).
- **Association** : `Souris` est associée à `Sexe` (1), `Etat` (1), `Caractere` (1) ; `Interaction` est associée à `TypeInteraction` (1) ; `Obstacle` est associé à `TypeObstacle` (1) ; `Environnement` est associé à `Configuration` (1).
- **Héritage** (flèche creuse) : `Souris`, `Nourriture` et `Obstacle` héritent de la classe abstraite `Element` ; `Case` peut contenir 0 ou 1 `Element`.
- **Naissance** (classe association, multiplicité 3) : la classe `Naissance` relie deux `Souris` (parent1, parent2) à l'enfant produit.

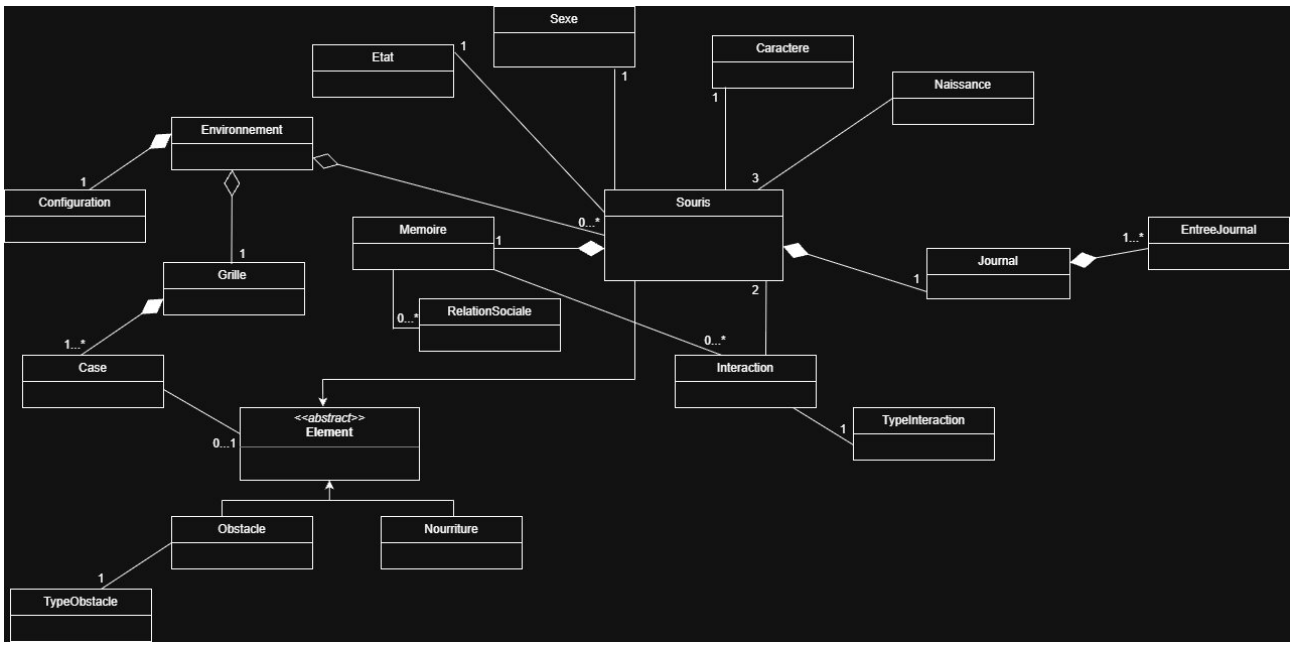


FIGURE 2 – Diagramme de classes simplifié — relations structurelles entre les 17 classes

3.3.2. Diagramme complet — attributs et méthodes

La figure 3 présente le diagramme de classes complet avec l'ensemble des attributs et méthodes de chaque classe. Voici les points saillants :

- `Souris` est la classe centrale : elle possède 9 attributs (`id`, `sexe`, `age`, `energie`, `caractere`, `etat`, `position`, `memoire`, `journal`) et 10 méthodes dont `vieillir()`, `perdreEnergie()`, `mourir()`, `ecrireJournal()` et `memoriserCase()`.
- `Environnement` est le moteur de simulation : il orchestre les 8 étapes de `nextRound()` via `ajouterSouris()`, `retirerSouris()` et `avancerTemps()`.
- `Memoire` centralise les connaissances d'une souris : `casesNourritureConnues`, `casesDangereuses`, `relationsSociales` et `interactionsPassees` sont toutes des listes.
- `Caractere` pilote le comportement social : les booléens `partageInformation` et `tendanceMensonge` déterminent si une souris Donneuse est sincère ou menteuse.
- **Classes énumération** : `Sexe`, `Etat`, `TypeObstacle` et `TypeInteraction` sont des enums avec méthodes utilitaires (`estCompatible()`, `estStable()`, `calculerPenalite()`, `estSociale()`).

— `Grille` et `Case` forment la carte 2D : `Case` expose `estLibre()`, `accueillirElement()` et `liberer()` pour gérer l'occupation des cases.



FIGURE 3 – Diagramme de classes complet — attributs et méthodes de toutes les classes

3.4. Les design patterns appliqués

Cinq design patterns issus directement des programmes de démonstration du professeur ont été appliqués. Chacun résout un problème de conception concret dans notre projet.

3.4.1. Pattern Template Method (tree_v1) — `nextRound()`

Le pattern Template Method définit le squelette d'un algorithme dans une méthode de base, en déléguant certaines étapes à des sous-classes ou à des méthodes auxiliaires.

Dans notre projet, `Environnement.nextRound()` définit le squelette de chaque tour de simulation. L'ordre des étapes est fixe et garanti.

```
1  public void nextRound() {
2      time++;
3      // Etape 1 : deplacement de toutes les souris
4      for (Souris s : mice) {
5          if (s.estVivante()) deplacerSouris(s);
6      }
7      // Etape 2 : gestion de la reproduction
8      gererReproduction();
9      // Etape 3 : gestion des interactions D->R
10     gererInteractions();
11     // Etape 4 : suppression des souris mortes
12     supprimerSourisMortes();
13     // Etape 5 : verification des conditions de fin
14     verifierConditionsFin();
15     // Etape 6 : mise a jour du tresor
16     gererTresor();
17     // Etape 7 : calcul du score de colonisation
18     calculerScoreColonisation();
19     // Etape 8 : enregistrement des statistiques
20     stats.enregistrer(time, mice, foods);
21 }
```

Listing 1 – Squelette du Template Method nextRound()

Ce pattern garantit que les étapes s'exécutent toujours dans le bon ordre. Par exemple, les interactions ont lieu après les déplacements, et les souris mortes sont retirées avant le calcul des statistiques.

3.4.2. Pattern Strategy (tree_v2) — `PaintStrategy`

Le pattern Strategy définit une famille d'algorithmes interchangeables et encapsule chacun d'eux. Dans notre projet, `PaintStrategy` encapsule la stratégie de rendu graphique de chaque case de la grille. Selon le contenu de la case (souris, nourriture, obstacle, case vide), une stratégie de dessin différente est appliquée.

Ce pattern permet d'ajouter de nouveaux types visuels (par exemple, le trésor) sans modifier `GameDisplay`. Il suffit d'ajouter une nouvelle stratégie de dessin dans `PaintStrategy`. Les sprites PNG sont chargés via `getResourceAsStream` depuis le classpath, garantissant leur portabilité entre machines.

3.4.3. Pattern Builder (tree_v1) — `GameBuilder`

Le pattern Builder sépare la construction d'un objet complexe de sa représentation. Dans notre projet, `GameBuilder.buildInitMobile()` construit l'état initial complet de la simulation : il crée la grille, place les souris, les nourritures et les obstacles aléatoirement, puis retourne un `Environnement` prêt à être utilisé.

Ce découplage permet à `MainGUI` de lancer la simulation avec un simple `manager = GameBuilder.buildInitMobi` sans connaître les détails d'initialisation. Il facilite également la reconfiguration : lorsque l'utilisateur modifie les paramètres et clique sur "Lancer", `GameBuilder` reconstruit entièrement la simulation.

3.4.4. Pattern Singleton (tree_v1) — `JournalFenetre`

Le pattern Singleton garantit qu'une classe n'a qu'une seule instance et fournit un point d'accès global à cette instance. Dans notre projet, `JournalFenetre` est un Singleton : une seule fenêtre de journal est ouverte à la fois, peu importe combien de souris existent dans la simulation.

L'accès se fait via `JournalFenetre.getInstance()`, qui crée la fenêtre si elle n'existe pas encore, ou retourne l'instance existante. Cela garantit la cohérence de l'affichage : quand l'utilisateur clique sur une souris différente, le journal se met à jour dans la même fenêtre plutôt que d'en ouvrir une nouvelle.

3.4.5. Pattern Simple Factory (tree_v1) — `addFood()`, `addObstacle()`

Le pattern Simple Factory centralise la création d'objets d'une famille donnée. Dans notre projet, `Environnement.addFood()` et `Environnement.addObstacle()` jouent le rôle de fabriques simples. La méthode `addFood()` crée un objet `Nourriture` avec un type choisi aléatoirement parmi les trois types disponibles (fromage, pomme, graines) et le place sur une case libre. L'IHM n'a pas besoin de connaître les types de nourriture : elle appelle simplement `addFood()`.

3.5. La boucle principale de simulation

La boucle principale est implémentée dans `MainGUI` via un thread dédié (séparé de l'EDT Swing), conformément aux bonnes pratiques des applications multi-threadées en Java.

```
new Thread(() -> {
    while (running) {
        if (!paused) {
            if (manager.isSimTerminee()) {
                paused = true;
                SwingUtilities.invokeLater(() -> dashboard.repaint());
                continue;
            }
            manager.nextRound();
            SwingUtilities.invokeLater(() -> {
                dashboard.repaint();
                statsPanel.repaint();
            });
        }
        try {
            Thread.sleep(vitesse); // vitesse en ms, controlable
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}).start();
```

Listing 2 – Boucle principale dans MainGUI (thread dédié)

La mise à jour de l'interface graphique est toujours effectuée dans l'Event Dispatch Thread (EDT) via `SwingUtilities.invokeLater()`, ce qui évite les problèmes de concurrence. Le paramètre `vitesse` (en millisecondes) est contrôlable par les boutons Vitesse+/Vitesse−, permettant d'accélérer ou de ralentir la simulation en temps réel.

Quand la simulation se termine (par victoire, SAD END ou convergence), le flag `paused` est mis à true et le thread s'arrête d'appeler `nextRound()`. L'interface reste affichée sur l'écran de fin jusqu'au clic de l'utilisateur sur le bouton de relance.

3.6. Le moteur de simulation : `Environnement.java`

La classe `Environnement` est le cœur du projet. Elle implémente `MobileInterface` et contient toute la logique de simulation : déplacement des agents, gestion des interactions, reproduction, cycles évolutifs, trésor et calcul du score.

3.6.1. Le déplacement des souris

Pour chaque souris vivante, `Environnement` appelle `choisirProchaineCaseAvecAttraction()` qui implémente l'algorithme des 8 priorités. Les cases candidates sont d'abord filtrées par l'anti-boucle, puis par les conditions de chaque priorité.

```
// Exclure les cases recemment visitees (anti-boucle)
Deque<String> recents = historiquePositions
    .getOrDefault(mouse.getId(), new ArrayDeque<String>());
List<Case> libresNouveaux = new ArrayList<Case>();
for (Case v : libres) {
    String k = v.getLine() + "," + v.getColumn();
    if (!recents.contains(k)) libresNouveaux.add(v);
}
// Utiliser les cases non-visitees si disponibles
List<Case> candidats = !libresNouveaux.isEmpty()
```

```
        ? libresNouveaux : libres;
// Apres deplacement, mettre a jour l'historique
if (!historiquePositions.containsKey(mouse.getId())) {
    historiquePositions.put(mouse.getId(),
        new ArrayDeque<String>());
}
historiquePositions.get(mouse.getId()).addLast(posKey);
// Garder seulement les 6 derni res positions
if (historiquePositions.get(mouse.getId()).size() > 6)
    historiquePositions.get(mouse.getId()).pollFirst();
```

Listing 3 – Mécanisme anti-boucle par historique de positions

3.6.2. La gestion des interactions

La méthode `gererInteractions()` parcourt toutes les paires de souris adjacentes et détermine le type d'interaction. Pour chaque paire, elle vérifie si un échange D→R peut avoir lieu (conditions : énergie pleine, caractère éligible, paire non déjà vue). Si oui, l'échange se produit et un flash est créé.

3.6.3. Le calcul du score de colonisation

Le score de colonisation est calculé à chaque tour selon l'équation 2. Il mesure quatre critères : la couverture de la grille (C = ratio cases occupées/cases accessibles), l'énergie moyenne normalisée (E = énergie_moy/MAX_ENERGY), la diversité des zones (D = ratio quadrants occupés/4) et la population relative (P = nb_vivantes/MAX_POPULATION).

```
private double calculerScoreColonisation() {
    // Couverture : cases distinctes occupees
    Set<String> casesOccupees = new HashSet<String>();
    for (Souris s : mice)
        if (s.estVivante())
            casesOccupees.add(s.getPosition().getLine()
                + "," + s.getPosition().getColumn());
    double couverture = (double) casesOccupees.size()
        / Math.max(1, totalCases - obstacles.size());

    // Energie moyenne normalisee
    double energieMoy = 0; int vivantes = 0;
    for (Souris s : mice)
        if (s.estVivante()) { energieMoy += s.getEnergie(); vivantes++; }
    energieMoy = vivantes > 0
        ? energieMoy / vivantes / Configuration.MAX_ENERGY : 0;

    // Diversite (4 quadrants)
    int[] quadrants = {0, 0, 0, 0};
    int midC = cols/2, midR = rows/2;
    for (Souris s : mice) {
        if (!s.estVivante()) continue;
        int q = (s.getPosition().getLine() < midR ? 0 : 2)
              + (s.getPosition().getColumn() < midC ? 0 : 1);
        quadrants[q]++;
    }
    int presents = 0;
    for (int q : quadrants) if (q > 0) presents++;
    double diversite = presents / 4.0;

    // Population normalisee
    double population = Math.min(1.0,
        (double) vivantes / Configuration.MAX_POPULATION);

    return couverture*40 + energieMoy*30
         + diversite*20 + population*10;
```

```
37 }
```

Listing 4 – Calcul du score de colonisation

3.7. Le système de mémoire en détail

La mémoire est implémentée dans la classe `Memoire` et constitue le mécanisme d'apprentissage des souris. Elle est mise à jour automatiquement par `Environnement` à chaque événement pertinent.

Lorsqu'une souris **trouve de la nourriture**, la case est ajoutée à `casesNourritureConnues`. Cette liste alimente directement la priorité 3 : à chaque tour, la souris peut choisir de retourner sur une case de nourriture connue plutôt que d'explorer aléatoirement. C'est un mécanisme d'exploitation vs. exploration.

Lorsqu'une souris **heurte un obstacle**, la case est ajoutée à `casesDangereuses`. Cette liste est consultée à la priorité 7 : la case est exclue des candidats de déplacement. Cela garantit qu'une souris n'heurte jamais deux fois le même obstacle — elle apprend de ses erreurs.

Lorsqu'une souris **reçoit une information** d'un Donneur, la case est stockée dans `caseCiblePartage`. Cette cible est consultée en priorité 1 — la priorité la plus haute après le trésor. Même si l'information est fausse (Donneur MECHANTE), la souris la suit aveuglément. C'est ce comportement de confiance qui rend les pièges possibles.

Lorsqu'une souris **rencontre une autre souris**, une `RelationSociale` est créée avec l'identifiant de l'autre souris et le type de relation (amitié, reproduction). Cette liste de relations est utilisée pour la condition d'accès au trésor (au moins 2 relations requises).

3.8. Le système d'interactions et les flashes visuels

Chaque interaction sociale génère un **flash visuel** affiché sur la grille pendant 3 tours. Un flash est une instance de `FlashInteraction` contenant les positions des deux souris concernées, le type d'interaction et un éventuel message textuel. La liste des flashes actifs est maintenue dans `Environnement.flashes` et mise à jour à chaque tour.

TypeInteraction	Visuel affiché	Condition de déclenchement
`AMITIE`	Bulle verte "amis !"	Rencontre entre deux souris compatibles
`PARTAGE_NOURRITURE`	Bulle jaune "info !" entre D et R	Échange D→R réussi
`REPRODUCTION`	Cœur rose ♡ entre les deux parents	Reproduction réussie
`PIEGE`	Nuage rouge "Oh ! Un piège !"	Receveur heurte un obstacle sur la cible partagée
`INDIFFERENCE`	Aucun visuel	Rencontre sans interaction possible

TABLE 11 – Types d'interactions et leurs représentations visuelles

Le rendu des flashes est assuré par `GameDisplay.drawFlashes()`, qui utilise un compositing alpha (`AlphaComposite`) pour dessiner les bulles et nuages avec transparence. Le flash PIEGE est un nuage rouge dessiné au-dessus de la souris piégée, avec le texte “Oh ! Un piège !” en blanc.

Le protocole complet du partage D→R est illustré dans la figure 3.8.

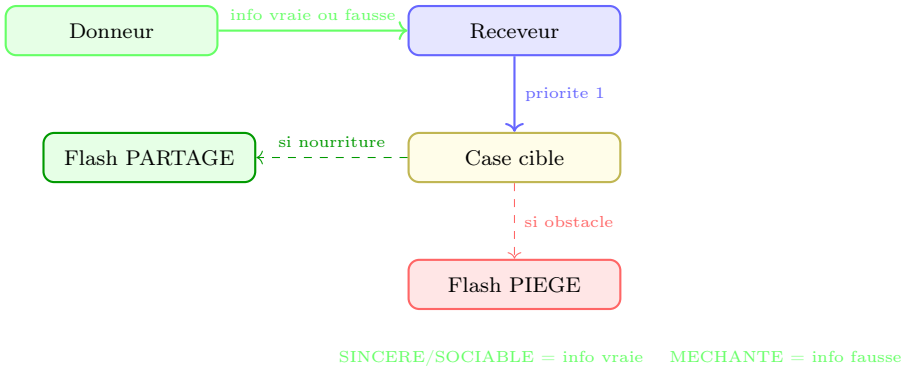


FIGURE 4 – Protocole complet du partage Donneur → Receveur

Protocole complet du partage Donneur → Receveur

3.9. La cible dynamique : implémentation du trésor

3.9.1. Algorithme de placement

Le trésor est placé sur le bord de la grille (première ou dernière ligne/colonne) à la position la plus éloignée de toutes les souris, calculée par distance de Manhattan. L’algorithme parcourt toutes les cases des bords, filtre les cases libres, puis choisit celle dont la distance minimale à toutes les souris est maximale.

```
// Collecter toutes les cases libres des bords
for (int c = 0; c < cols; c++) {
    bords.add(map.getBlock(0, c));
    bords.add(map.getBlock(rows - 1, c));
}
for (int l = 1; l < rows - 1; l++) {
    bords.add(map.getBlock(l, 0));
    bords.add(map.getBlock(l, cols - 1));
}
// Choisir la case la plus eloignee de toutes les souris
Case meilleur = null;
int maxDistMin = -1;
for (Case c : candidates) {
    int distMin = Integer.MAX_VALUE;
    for (Souris s : mice) {
        int d = Math.abs(s.getPosition().getLine()-c.getLine())
              + Math.abs(s.getPosition().getColumn()-c.getColumn());
        if (d < distMin) distMin = d;
    }
    if (distMin > maxDistMin) {
        maxDistMin = distMin;
        meilleur = c;
    }
}
```

Listing 5 – Placement du trésor sur le bord le plus éloigné

3.9.2. Rendu visuel du trésor

Le trésor a deux états visuels distincts dans `GameDisplay.drawTresor()`, selon que la simulation est avant ou après le tour 100.

Avant le tour 100 (trésor scellé). Le fond de la case est grisé (`Color(100, 90, 70)`). L'image du coffre est affichée avec une transparence de 35% (`AlphaComposite` à 0.35). Un cadenas doré est dessiné au centre de la case. Un compte à rebours "T-X" est affiché en bas de la case, où X est le nombre de tours restants avant l'ouverture.

Après le tour 100 (trésor accessible). Le fond de la case devient jaune doré (`Color(255, 230, 50)`). Un halo pulsant est animé grâce à la fonction sinus : `Math.sin(System.currentTimeMillis() / 350.0)`, ce qui crée un effet de pulsation naturel. L'image du coffre est affichée en pleine opacité. Le label "TRESOR" est affiché en bas de la case.

3.9.3. Les trois écrans de fin

Chacune des trois fins de simulation donne lieu à un écran dessiné entièrement en Java2D par `GameDisplay` :

Victoire. L'écran est doré (`Color(20, 15, 5)` avec un halo `Color(255, 200, 0, 50)`). Il affiche le titre "TRÉSOR TROUVÉ !", l'identité complète de la gagnante (ID, caractère, sexe, couleur, énergie, âge, mémoire), les statistiques finales (tour, génération, survivants) et un message "Elle restera dans les mémoires de la colonie pour toujours !". Le bouton "Nouvelle aventure" est cliquable.

SAD END. L'écran est rouge sombre (`Color(10, 5, 5)`). Il affiche "La colonie s'est éteinte...", un message poétique et un bouton "Nouvelle colonie (3 souris)" qui relance la simulation avec exactement 3 nouvelles souris.

Colonisation optimale. L'écran affiche une note de D à S+ selon le score final, une barre de progression visuelle, les statistiques finales et un message adapté à la performance.

3.10. L'IHM Graphique en détail

La figure 5 présente la vue d'ensemble de la fenêtre principale. Les figures suivantes détaillent chaque composant.

3.10.1. Vue globale de la fenêtre principale

La fenêtre principale `MainGUI` est organisée selon un `BorderLayout` en trois zones fixes. À l'**ouest**, le `StatsPanel` affiche les statistiques dynamiques de la simulation. Au **centre**, le `GameDisplay` affiche la grille animée. À l'**est**, le panneau de contrôle contient huit boutons d'action. Une fenêtre contextuelle non-modale, `JournalFenetre`, s'ouvre au clic sur une souris.

3.10.2. Détail : le StatsPanel

Le `StatsPanel` est dessiné entièrement en Java2D, sans librairie externe. Il contient cinq blocs mis à jour à chaque appel de `nextRound()` : les compteurs rapides, la courbe de population et de nourriture, la courbe d'énergie moyenne, le camembert des types d'interactions et les formules de probabilité avec le score de colonisation courant.

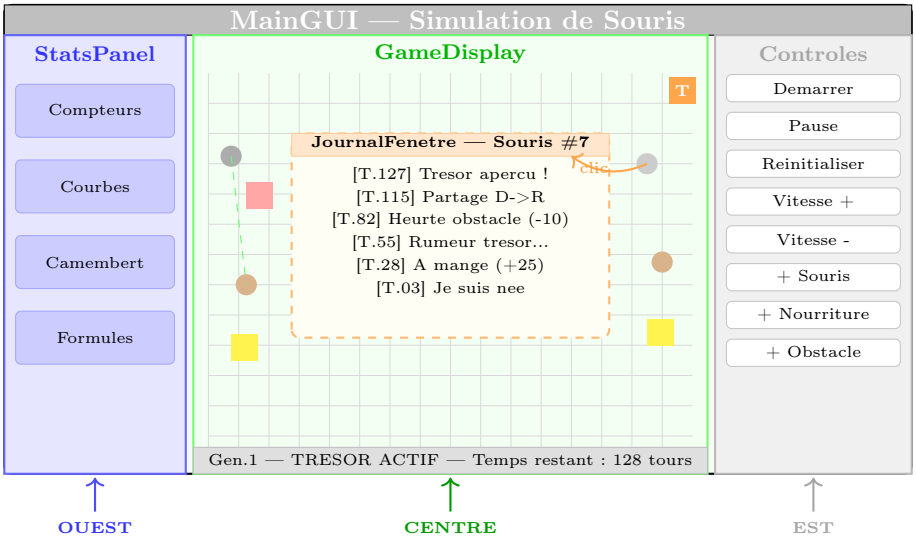


FIGURE 5 – Vue globale de la fenêtre principale — MainGUI (BorderLayout)

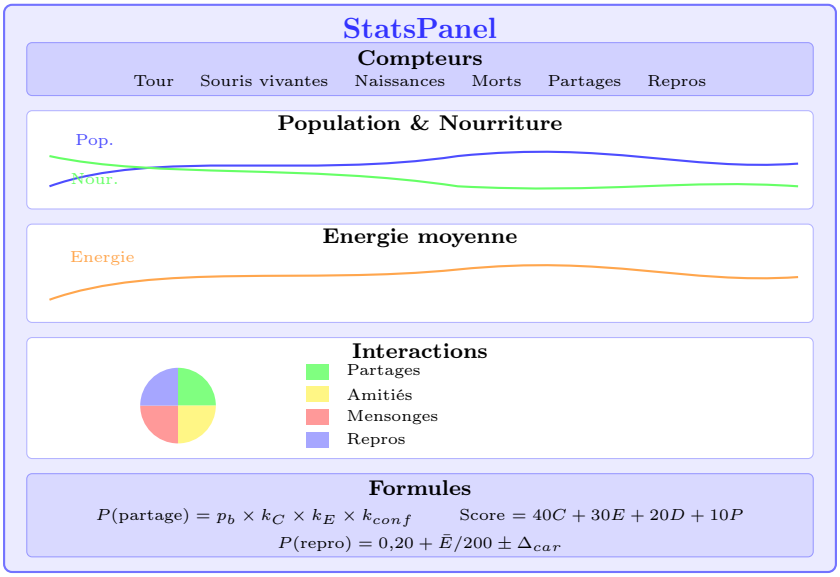


FIGURE 6 – Détail du StatsPanel — 5 blocs mis à jour à chaque tour

3.10.3. Détail : le GameDisplay

Le `GameDisplay` a deux modes de fonctionnement. Au démarrage, il affiche l'overlay de configuration permettant de régler les paramètres avec des boutons [−]/[+]. Après le clic sur "Lancer", il affiche la grille de simulation en temps réel avec les souris, nourritures, obstacles, flashes et le trésor. La barre d'état en bas affiche le tour courant, le score et l'état du trésor.

3.10.4. Détail : la JournalFenetre

La `JournalFenetre` est une `JDialog` non-modale qui s'ouvre lorsque l'utilisateur clique sur une souris dans la grille. Elle est implémentée comme un Singleton : une seule instance existe, et son contenu se met à jour selon la souris cliquée. Elle affiche l'identité complète de la souris en en-tête, puis son journal intime en ordre anti-chronologique avec un code couleur par type d'événement.

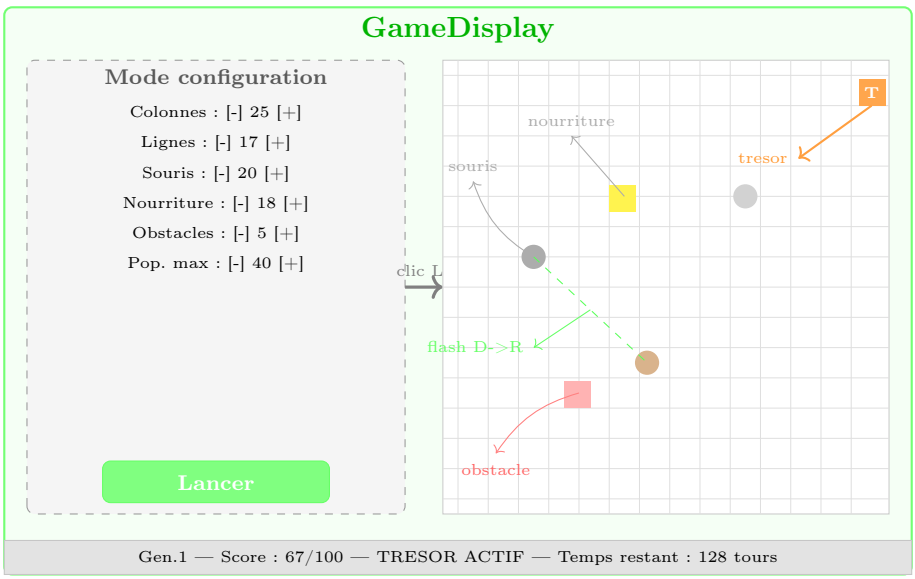


FIGURE 7 – Détail du GameDisplay (overlay config. à gauche, grille de simulation à droite)

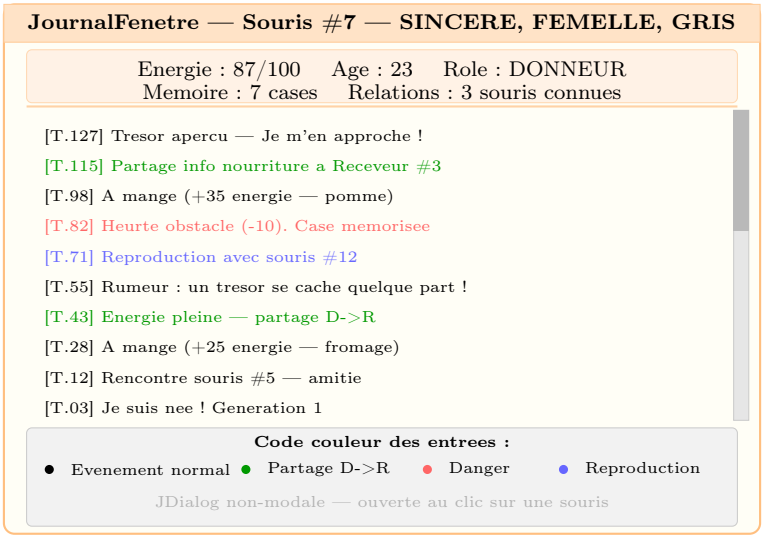


FIGURE 8 – Détail de la JournalFenetre (fenêtre contextuelle par souris)

3.11. Journalisation avec Log4j

La journalisation est assurée par le framework **Log4j 1.2.17**, fourni par M. Liu dans le cadre du TD 6. La classe `LoggerUtility` (identique à *tree_ v3*) offre deux modes de sortie : texte (pour la console et le fichier `log/souris-log.txt`) et HTML (pour le fichier `log/souris-log.html`).

La différence avec la version originale de *tree_ v3* réside dans le chargement des fichiers de configuration. Dans notre version, les `.properties` sont chargés via `getResourceAsStream` depuis le classpath plutôt que par un chemin fichier relatif. Cela garantit que les logs fonctionnent correctement quelle que soit la façon dont Eclipse lance le programme.

```
1  public static Logger getLogger(Class<?> logClass,
2                                 String logFileType) {
3      try {
4          String configFile = logFileType.equals("html")
5              ? "log/log4j-html.properties"
```

```
            : "log/log4j-text.properties";
        InputStream is = LoggerUtility.class
            .getClassLoader()
            .getResourceAsStream(configFile);
        if (is != null) {
            Properties props = new Properties();
            props.load(is);
            PropertyConfigurator.configure(props);
        }
    } catch (Exception e) {
        System.err.println("Log4j config error: "
            + e.getMessage());
    }
    return Logger.getLogger(logClass.getName());
}
```

Listing 6 – Chargement des properties Log4j via le classpath

Les événements loggés couvrent l'ensemble du cycle de vie de la simulation : création des souris, déplacements significatifs, interactions D→R, reproductions, morts, apparition du trésor, conditions de fin et score final.

4. Manuel utilisateur

Cette section décrit le fonctionnement complet du logiciel du point de vue de l'utilisateur final.

4.1. Environnement de développement

Le projet a été développé et testé sous **Eclipse IDE for Java Developers** avec **Java 8 (JDK 1.8)**. Le projet est livré sous forme d'un dossier `ProjetSouris/` contenant les sources, les librairies et les fichiers de configuration Eclipse (`.classpath`, `.project`, `.settings/`).

Les cinq librairies externes sont incluses dans le dossier `libs/` et référencées automatiquement par Eclipse via le `.classpath`. Aucune installation supplémentaire n'est requise : le projet est directement opérationnel après import dans Eclipse via `File` → `Import` → `Existing Projects into Workspace`.

4.2. Configuration initiale de la simulation

Au lancement, la fenêtre principale s'ouvre avec l'overlay de configuration affiché sur la grille. L'utilisateur peut régler les paramètres à l'aide des boutons `[-]` et `[+]` de chaque ligne. Le tableau 12 récapitule les paramètres disponibles.

Paramètre	Minimum	Maximum	Valeur par défaut
Nombre de colonnes	5	40	25
Nombre de lignes	5	30	17
Nombre de souris	1	30	20
Nombre de nourritures	1	30	18
Nombre d'obstacles	0	20	5
Population maximale	5	80	40

TABLE 12 – Paramètres de configuration disponibles au démarrage

Une fois satisfait de la configuration, cliquer sur le bouton **Lancer la simulation**. L'overlay disparaît et la grille de simulation s'affiche en temps réel.

4.3. Contrôle de la simulation en cours

Le panneau de contrôle à droite de la fenêtre contient huit boutons. Leur rôle est détaillé dans le tableau 13.

Bouton	**Action**
Démarrer	Lance ou reprend la simulation après une pause
Pause	Met la simulation en pause. Cliquer sur Démarrer pour reprendre
Réinitialiser	Repart de zéro avec la même configuration (toutes les souris, nourritures, obstacles recréés)
Vitesse +	Réduit le délai entre les tours, accélère la simulation
Vitesse −	Augmente le délai entre les tours, ralentit la simulation
+ Souris	Ajoute une souris avec des attributs aléatoires sur une case libre
+ Nourriture	Ajoute une nourriture aléatoire (fromage, pomme ou graines) sur une case libre
+ Obstacle	Ajoute un obstacle aléatoire (bûche, rivière ou rocher) sur une case libre

TABLE 13 – Boutons de contrôle et leurs actions

4.4. Lecture des statistiques

Le `StatsPanel` à gauche de la fenêtre affiche en temps réel cinq types d'informations :

- **Compteurs** : le tour courant, le nombre de souris vivantes, le nombre cumulé de naissances, de morts, de partages D→R et de reproductions depuis le début de la simulation.
- **Courbe Population/Nourriture** : l'évolution du nombre de souris vivantes (bleu) et du nombre de nourritures disponibles (vert) au fil des tours.
- **Courbe Énergie** : l'évolution de l'énergie moyenne de la colonie (orange). Une baisse indique que les souris peinent à trouver de la nourriture.
- **Camembert des interactions** : la proportion de chaque type d'interaction depuis le début (partages en vert, amitiés en jaune, mensonges en rouge, reproductions en bleu).
- **Formules et score** : les formules de probabilité utilisées par le moteur et le score de colonisation courant, calculé selon l'équation 2.

4.5. Consulter le journal d'une souris

Un clic sur n'importe quelle souris visible dans la grille ouvre (ou met à jour) la `JournalFenetre`. Cette fenêtre affiche :

- **En-tête** : l'identité complète de la souris (ID, caractère, sexe, couleur, énergie actuelle, âge, rôle, nombre de cases mémorisées et nombre de relations sociales).
- **Journal** : toutes les entrées du journal intime de la souris, en ordre anti-chronologique (la plus récente en haut), avec un code couleur par type d'événement.

4.6. Suivre l'évolution du trésor

À partir du **tour 50**, un coffre apparaît sur le bord de la grille. Il est affiché en grisé avec un cadenas doré et un compte à rebours visible dans la case. La barre d'état en bas du `GameDisplay` affiche le message *"Trésor scellé — s'ouvre au tour 100"*.

À partir du **tour 100**, la case du trésor change d'apparence : elle devient jaune dorée avec un halo pulsant animé et le label "TRESOR". La barre d'état affiche désormais *"TRÉSOR ACTIF — Temps restant : X tours"*, où X décroît à chaque tour jusqu'à 300.

Si une souris atteint le trésor et remplit les conditions (énergie ≥ 85, âge ≥ 15, mémoire ≥ 5 cases, ≥ 2 relations), la simulation se termine immédiatement avec l'écran de victoire doré.

4.7. Interprétation des fins de simulation

La simulation peut se terminer de trois façons, chacune donnant lieu à un écran de fin spécifique avec un bouton de relance :

Victoire (écran doré). Une souris a réussi à atteindre le trésor entre les tours 100 et 300 en remplissant toutes les conditions. L'écran affiche son identité complète et le message "Elle restera dans les mémoires de la colonie pour toujours !" Le bouton "Nouvelle aventure" relance une nouvelle simulation.

SAD END (écran rouge). Toutes les souris sont mortes sans que le trésor n'ait été trouvé. Cela peut arriver parce que la fenêtre de recherche 100–300 s'est fermée sans succès, et la simulation a continué jusqu'à l'extinction totale de la colonie. Le bouton "Nouvelle colonie (3 souris)" relance la simulation avec seulement 3 nouvelles souris.

Colonisation optimale (écran bleu). Le score de colonisation s'est stabilisé pendant 10 tours consécutifs, indiquant que le système a atteint un équilibre. L'écran affiche une note de D à S+ selon le score final et une barre de progression. Le bouton "Nouvelle simulation" relance avec les mêmes paramètres.

5. Déroulement du projet

5.1. Organisation du travail en équipe

Le projet a été réalisé en équipe de trois personnes sur plusieurs semaines. Nous avons adopté une organisation de travail itérative et collaborative : chaque fonctionnalité était discutée ensemble, implémentée par l'une d'entre nous, puis revue et testée par les deux autres. Cette approche nous a permis de maintenir une cohérence dans le code et de détecter rapidement les erreurs.

Le tableau 14 présente la répartition principale des responsabilités au sein de l'équipe.

5.2. Problèmes rencontrés et solutions apportées

Le développement d'un projet de cette envergure a inévitablement donné lieu à de nombreux problèmes techniques. Nous décrivons ici les huit problèmes les plus significatifs rencontrés, ainsi que les solutions que nous avons apportées.

Membre	Responsabilités principales
SIMYA Meriem	Moteur de simulation (`Environnement.java`), algorithme de déplacement à 8 priorités, système de mémoire, cycles évolutifs
LAIFAOUI Ines	Interface graphique (`GameDisplay.java`, `StatsPanel.java`), système de flashes visuels, `PaintStrategy`, sprites PNG
ANOUD Salma	Interactions D→R, système trésor, tests JUnit, journalisation Log4j, rapport LaTeX

TABLE 14 – Répartition du travail au sein de l'équipe

5.2.1. Problème 1 : Conflit de versions Java dans Eclipse

Lors des premiers tests, Eclipse compilait le projet avec une version Java 11 ou supérieure, alors que notre code était conçu pour Java 8. Ce conflit se manifestait par l'erreur `StringConcatFactory cannot be resolved` au lancement, qui bloquait entièrement l'exécution de la simulation.

Pour résoudre ce problème, nous avons ajouté le fichier `.settings/org.eclipse.jdt.core.prefs` dans le projet, avec les directives `compliance=1.8`, `source=1.8` et `targetPlatform=1.8`. Nous avons également mis à jour le `.classpath` pour référencer explicitement le conteneur JRE JavaSE-1.8. Ces deux fichiers sont désormais inclus dans le projet livré, ce qui garantit que n'importe quel poste Eclipse compilera correctement.

5.2.2. Problème 2 : Images des sprites non trouvées au lancement

Dans une première version, `PaintStrategy` cherchait les sprites de souris via un chemin relatif simple (par exemple `"souris_gris_male_right.png"`). Ce chemin n'était pas résolu correctement par Eclipse, qui cherchait les images dans le répertoire courant d'exécution plutôt que dans le répertoire `src/resources/`.

La solution a consisté à utiliser `getClass().getClassLoader().getResourceAsStream("resources/souris_xx` pour charger les images depuis le classpath. Étant donné que le dossier `src/` est configuré comme source folder dans le `.classpath`, le répertoire `src/resources/` est automatiquement inclus dans le classpath, et les images sont trouvées quelle que soit la façon dont Eclipse lance le programme.

5.2.3. Problème 3 : Fichiers de configuration Log4j non trouvés

Dans la version initiale, `LoggerUtility` chargeait les fichiers `.properties` via un chemin fichier relatif (`"src/log/log4j-text.properties"`). Ce chemin dépendait du répertoire courant au moment du lancement, ce qui causait l'erreur `FileNotFoundException` lorsque le programme était lancé depuis un répertoire différent de la racine du projet.

Nous avons modifié `LoggerUtility` pour charger les properties via `getResourceAsStream` depuis le classpath (`"log/log4j-text.properties"`), exactement comme pour les sprites. Les fichiers `.properties` étant dans `src/log/`, ils sont bien présents dans le classpath et toujours trouvés, quelle que soit la configuration Eclipse.

5.2.4. Problème 4 : Boucles infinies de déplacement des souris

Sans précaution, les souris se retrouvaient fréquemment bloquées dans des boucles de déplacement entre deux cases adjacentes, rendant la simulation statique et peu intéressante à observer. Ce phéno-

mène était particulièrement visible quand plusieurs souris se trouvaient dans une zone avec peu de cases libres.

Nous avons implémenté un mécanisme anti-boucle basé sur un historique des 6 dernières positions de chaque souris. Avant de choisir une case candidate, les cases présentes dans l'historique sont exclues de la liste des candidats. Ce mécanisme est implémenté via une `Deque<String>` par souris dans la table de hachage `historiquePositions`. Si toutes les cases accessibles sont dans l'historique (situation de blocage total), le filtrage est désactivé pour permettre quand même le déplacement.

5.2.5. Problème 5 : Souris qui heurtent les mêmes obstacles répétitivement

Dans une version préliminaire, les souris pouvaient heurter le même obstacle plusieurs fois par tour, perdant de l'énergie de façon répétée et mourant prématurément. Ce comportement était dû à l'absence de mémorisation des cases dangereuses : une souris essayait de se déplacer vers un obstacle, échouait, puis recommençait le tour suivant.

La solution a été d'enrichir la classe `Memoire` avec une liste `casesDangereuses`. Chaque fois qu'une souris tente de se déplacer vers un obstacle, la case est immédiatement ajoutée à `casesDangereuses` et sera exclue de toutes les futures décisions de déplacement (priorité 7). Ce mécanisme simule l'apprentissage par l'expérience négative.

5.2.6. Problème 6 : Fichier GameDisplay.java vidé par erreur d'encodage

Lors d'une modification automatique du fichier `GameDisplay.java` via un script Python, une erreur d'encodage UTF-8 a causé la troncature puis le vidage complet du fichier. Ce fichier, qui contient plus de 600 lignes de code, se retrouvait entièrement vide, ce qui bloquait totalement la compilation.

Nous avons restauré le fichier depuis la sauvegarde disponible dans le dossier de backup, puis réintégré manuellement toutes les fonctionnalités ajoutées après la sauvegarde (trésor, écrans de fin, flashes PIEGE, score en barre d'état). Cette expérience nous a enseigné l'importance de sauvegardes régulières et de la gestion de version.

5.2.7. Problème 7 : Trésor inaccessible (entouré d'obstacles)

Dans une première version du système trésor, nous avions placé automatiquement 2 obstacles autour de la case trésor pour le "protéger" et le rendre plus difficile d'accès. En pratique, cette approche rendait le trésor totalement inaccessible : les souris ne pouvaient pas l'atteindre, car elles devaient contourner les obstacles tout en maintenant une énergie ≥ 85, ce qui était impossible dans la plupart des configurations.

Nous avons supprimé la génération automatique d'obstacles autour du trésor et simplifié les conditions d'accès à des valeurs réalistes mais exigeantes : énergie ≥ 85, âge ≥ 15, mémoire ≥ 5 cases, relations ≥ 2 souris. Ces conditions garantissent que le trésor est difficile mais atteignable.

5.2.8. Problème 8 : Redémarrage automatique indésirable

Une version intermédiaire du projet implémentait un redémarrage automatique de la simulation après 5 secondes à la fin. Ce comportement était gênant : l'utilisateur n'avait pas le temps de lire l'écran de fin (identité de la gagnante, statistiques finales) avant que la simulation ne reparte.

Nous avons supprimé le thread de redémarrage automatique. Désormais, la simulation se met en pause sur l'écran de fin et attend que l'utilisateur clique manuellement sur le bouton de relance. Ce bouton est cliquable (détecté dans le `MouseListener` de `GameDisplay`) et déclenche un appel à `manager.reset()` suivi d'une réinitialisation de l'interface.

5.3. Les tests JUnit

Les tests automatisés sont une part essentielle de la qualité du projet. Nous avons utilisé le framework **JUnit 4.11** fourni par M. Liu dans le cadre du TD 5. Cinq classes de tests couvrent les composants principaux du projet.

Classe de test	Cas de tests couverts
`TestSouris`	Création d'une souris, énergie initiale correcte, décrémentation d'énergie, mort quand énergie = 0, incrémentation de l'âge
`TestInteractions`	Partage D→R (SINCERE donne vraie info), partage (MECHANTE donne fausse info), reproduction (conditions remplies), flash PIEGE (obstacle sur cible partagée)
`TestEnvironnement`	`nextRound()` (tour incrémenté), `addFood()` (nourriture ajoutée), `addObstacle()` (obstacle ajouté), `reset()` (état réinitialisé), convergence (score stable)
`TestGame`	Test d'intégration : lancement de la simulation, exécution de 10 tours, vérification que les souris sont vivantes et la grille cohérente
`TestSuiteAll`	Suite JUnit regroupant les quatre classes de tests ci-dessus en une seule exécution

Table 15 – Classes de tests JUnit et cas couverts

Chaque test vérifie un comportement atomique et indépendant. Par exemple, `TestSouris` vérifie qu'une souris créée avec énergie 80 a bien 80 points après création, qu'après 5 appels à `perdreEnergie(1)` elle a 75 points, et qu'après 80 appels supplémentaires elle est marquée comme morte (`estVivante() == false`).

Les tests `TestInteractions` sont particulièrement importants car ils vérifient les comportements sociaux qui sont au cœur de la simulation. Ils s'assurent notamment qu'une souris SINCERE transmet bien la vraie position de la nourriture, qu'une souris MECHANTE transmet bien une fausse position, et qu'un flash PIEGE est bien créé quand le Receveur heurte l'obstacle indiqué.

La suite `TestSuiteAll` permet d'exécuter tous les tests en une seule fois via un clic droit → Run As → JUnit Test sur la classe `TestSuiteAll.java`.

6. Conclusion

6.1. Bilan technique

Ce projet nous a permis de concevoir et d'implémenter une simulation multi-agents complète en Java, en appliquant rigoureusement les principes du génie logiciel enseignés en cours. Le résultat est un logiciel fonctionnel, configurable, visuellement riche et documenté.

Sur le plan architectural, nous avons respecté la séparation en couches préconisée par M. Liu, avec une dépendance strictement descendante entre les cinq couches du projet. L'interface `MobileInterface` assure le découplage entre l'IHM et le moteur, ce qui nous a permis de modifier l'un sans impacter l'autre à plusieurs reprises pendant le développement.

Sur le plan des design patterns, les cinq patterns appliqués se sont révélés être de véritables outils de conception et non de simples exercices académiques. Le Template Method a structuré la boucle de simulation de façon claire et extensible. Le Strategy a rendu le rendu graphique modulaire. Le Builder a simplifié la reconfiguration au démarrage. Le Singleton a garanti la cohérence du journal. La Simple Factory a centralisé la création des objets du jeu.

Le tableau 16 récapitule les principales métriques du projet.

Métrique	**Valeur**
Nombre de packages	7
Nombre de fichiers Java	36
Nombre de classes/interfaces	34
Nombre de design patterns appliqués	5
Nombre de classes de tests JUnit	5
Nombre de librairies externes utilisées	5
Niveaux de priorité du déplacement	8
Types d'interactions possibles	5
Types de fins de simulation	3
Conditions d'accès au trésor	4

TABLE 16 – Métriques du projet

6.2. Difficultés surmontées

Le principal défi de ce projet était la coordination entre les trois membres de l'équipe pour maintenir la cohérence d'un code réparti entre 36 fichiers. Nous avons surmonté ce défi grâce à une communication régulière, une répartition claire des responsabilités et une relecture systématique du code produit par chaque membre.

Sur le plan technique, les problèmes les plus difficiles à résoudre ont été la gestion des versions Java dans Eclipse, le chargement des resources depuis le classpath, et la prévention des boucles infinies de déplacement. Ces problèmes nous ont obligées à approfondir notre compréhension de l'environnement Java et d'Eclipse, ce qui constitue un apprentissage précieux.

6.3. Perspectives d'amélioration

Plusieurs fonctionnalités pourraient enrichir le projet dans une version future :

- **Sauvegarde et chargement** : sérialisation Java pour sauvegarder l'état complet de la simulation et la recharger ultérieurement.
- **Évolution génétique** : les descendants héritent non seulement de la couleur mais aussi du caractère de leurs parents, avec une probabilité de mutation. Cela introduirait une vraie dynamique évolutive.
- **Plusieurs trésors** : plusieurs cibles avec des niveaux de difficulté différents et des conditions d'accès variées. La première souris à en atteindre un gagnerait, mais les autres resteraient disponibles.
- **Obstacles dynamiques** : des obstacles qui se déplacent (par exemple une rivière qui change de lit), forçant les souris à adapter leur mémoire des cases dangereuses.
- **Simulation réseau** : distribution de la simulation sur plusieurs machines avec partage des agents via sockets Java.
- **Interface de configuration avancée** : permettre de modifier les poids de l'algorithme de déplacement en temps réel, pour expérimenter différentes stratégies.

6.4. Apprentissages

Ce projet nous a appris à structurer un projet Java de taille conséquente en suivant les principes du génie logiciel. Nous avons compris concrètement pourquoi la séparation IHM/Moteur est importante : elle nous a permis de modifier l'interface graphique sans toucher au moteur, et inversement, à de nombreuses reprises pendant le développement.

L'application des design patterns nous a montré que ces patrons ne sont pas des constructions abstraites mais des solutions éprouvées à des problèmes récurrents. Chaque pattern que nous avons appliqué a résolu un problème concret de conception et amélioré la maintenabilité du code.

Enfin, les tests JUnit nous ont appris l'importance de tester chaque composant de façon indépendante. À plusieurs reprises, nos tests ont détecté des régressions introduites lors de modifications du code, ce qui nous a évité de livrer une version défectueuse.

Nous remercions encore M. Tianxiao Liu pour la qualité de son encadrement, la pertinence de ses programmes de démonstration, et sa disponibilité tout au long du semestre.

Références

[Apa12] Apache Software Foundation. Apache log4j 1.2.17, 2012. Fourni par M. Liu — TD 6 Journalisation.

[BG12] Kent Beck and Erich Gamma. Junit 4.11, 2012. Fourni par M. Liu — TD 5 Tests.

[GHJV94] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[Gil14a] David Gilbert. Jcommon 1.0.23, 2014.

[Gil14b] David Gilbert. Jfreechart 1.0.19, 2014. Fourni par M. Liu — TD Instrumentation.

[Ham12] Hamcrest Team. Hamcrest core 1.3, 2012.

[Liu26] Tianxiao Liu. Génie logiciel et projet — supports de cours et programmes, 2026. CY Cergy Paris Université, L2 Informatique. Programmes : Chronometer, tree_v1/v2/v3, Aircraft, Train. latex_demo.zip. `https://depinfo.u-cergy.fr/~tliu/glp.php`.

[Mar08] Robert C. Martin. *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

[Ove26] Overleaf Team. Overleaf — Éditeur latex collaboratif, 2026.